

DOCAYA

Enterprise Electronic Document Management System

Technical Architecture Document

Architecture · Development · Build · Infrastructure · Network · Deployment · Operations

Document type	Technical Architecture Document (TAD)
Platform	Microsoft Azure · Microsoft 365 · SharePoint Online · Microsoft Entra ID · National SSO
Architecture style	Cloud-native, event-driven, staging-first, zero-trust
Version	1.1 (adds development, build, infrastructure, network and deployment)
Status	Draft for Review
Classification	OFFICIAL – Internal
Date	September 2026

Document Control

Version History

Version	Date	Summary of change
0.1	Aug 2026	Initial architecture skeleton and principles
0.9	Sep 2026	Solution and application architecture, security and data views
1.0	Sep 2026	Complete technical baseline with all architecture aspects and diagrams
1.1	Sep 2026	Added development methodology (VS Code), repository & package structure, detailed application, infrastructure and network layers, and package deployment at the client site

Reviewers & Approvals

Review area	Responsibility	Decision
Solution Architecture	Architecture fitness, integration, operability	Pending
Security & Identity	Zero-trust, identity, secrets, network controls	Pending
Data & Compliance	Classification, retention, residency, audit	Pending
Platform & Operations	Reliability, observability, deployment	Pending
Programme Governance	Scope, risk, investment	Pending

Table of Contents

Document Control	2
Version History	2
Reviewers & Approvals	2
1. Introduction	6
1.1 Purpose	6
1.2 Scope	6
1.3 Intended Audience	6
1.4 Definitions & Acronyms	6
1.5 Requirement Conventions	6
2. Architecture Goals & Principles	7
2.1 Architecture Goals	7
2.2 Guiding Principles	7
2.3 Architecturally Significant Quality Attributes	7
2.4 Constraints & Assumptions	7
3. Architecture Approach & Views	8
4. Solution Architecture	9
4.1 Users & Channels	11
4.2 Identity & Single Sign-On	11
4.3 Security Edge	11
4.4 Application Services	11
4.5 Asynchronous Orchestration & Messaging	11
4.6 Data & Content	11
4.7 Microsoft 365 — System of Record	12
4.8 Cross-Cutting Platform	12
5. Application Architecture	13
5.1 Presentation Tier	15
5.2 API Gateway & Backend-for-Frontend	15
5.3 Application Services (Domain Modules)	15
5.4 Domain Model & Business Logic	15
5.5 Integration & Anti-Corruption Adapters	16
5.6 Data Access & Persistence	16
5.7 Cross-Cutting Concerns & Async Workers	16
6. Process & Runtime View — End-to-End Lifecycle	17
6.1 Consistency & Idempotency	17
7. Data Architecture	18
7.1 Storage Strategy	18
7.2 Core Entities	18
7.3 Data Classification & Handling	18
7.4 Retention, Legal Hold & Residency	18
8. Integration Architecture	19
8.1 SharePoint & Microsoft Graph	19
8.2 AI & Retrieval-Augmented Generation	19
8.3 Notifications	19
8.4 API & Contracts	19
9. Security Architecture	20

9.1 Identity & Access.....	20
9.2 Network & Perimeter.....	20
9.3 Data Protection & Content Safety.....	20
9.4 Audit & Tamper-Evidence.....	20
9.5 Threat Model (summary).....	20
10. Infrastructure & Deployment (Overview).....	21
10.1 Compute & Hosting.....	21
10.2 Environments.....	21
10.3 CI/CD & Release Gates.....	21
11. Observability & Operations.....	22
11.1 Telemetry.....	22
11.2 Monitoring & Alerting.....	22
11.3 SLOs & Runbooks.....	22
12. Non-Functional Requirements.....	23
13. Resilience, Availability & Disaster Recovery.....	23
14. Compliance & Governance.....	23
15. Technology Stack Summary.....	24
16. Architecture Decision Records.....	24
17. Risks & Mitigations.....	24
18. Development Methodology & Guidelines (VS Code).....	25
18.1 Delivery Approach.....	25
18.2 Separation into Independently Developed Packages.....	25
18.3 Developer Workstation Prerequisites.....	25
18.4 VS Code Workspace & Recommended Extensions.....	26
18.4.1 Multi-root workspace (docaya.code-workspace).....	26
18.4.2 Recommended extensions (.vscode/extensions.json).....	26
18.5 Getting Started — First Run in VS Code.....	26
18.6 Local Debug Profiles (.vscode/launch.json).....	26
18.7 Coding Standards & Quality Gates.....	27
18.8 Contract-First Development.....	27
19. Repository & Package Structure.....	28
19.1 Top-Level Layout.....	28
19.2 Web UI Package (src/web).....	28
19.3 API / BFF Package (src/api).....	28
19.4 Workers / Workflow Package (src/functions).....	28
19.5 Package Boundaries & Dependency Rules.....	28
20. Infrastructure Layer — IaC & Provisioning (Detailed).....	30
20.1 Bicep Module Structure (infra/).....	30
20.2 Resource Inventory (per environment).....	30
20.3 Provisioning Order & Dependencies.....	30
20.4 Identity, Secrets & Least-Privilege Roles.....	31
20.5 Validate & Deploy Commands.....	31
20.6 Environment Matrix.....	31
21. Network Architecture (Detailed).....	32
21.1 Topology.....	32
21.2 Private Connectivity & DNS.....	32
21.3 Ingress & Egress Controls.....	32

21.4 Traffic Flow (request to data).....	32
21.5 Network Security Controls.....	32
22. Package Deployment at the Client Site.....	34
22.1 What the Release Package Contains.....	34
22.2 Client-Side Prerequisites.....	34
22.3 Deployment Sequence.....	34
22.4 Configuration Binding (no secrets in artifacts).....	34
22.5 Release Strategy, Validation & Rollback.....	35
22.6 Upgrades & Patching.....	35
23. Assumptions, Open Items & Next Steps.....	36
23.1 Open Items.....	36
23.2 Next Steps.....	36
Appendices.....	37
Appendix A — CI/CD Pipeline (GitHub Actions).....	37
A.1 Pull-Request Validation — .github/workflows/pr.yml.....	37
A.2 Release & Deploy — .github/workflows/release.yml.....	38
Appendix B — Sample Infrastructure-as-Code (Bicep).....	40
B.1 Composition Root — infra/main.bicep.....	40
B.2 Example Module — modules/containerapp.bicep (excerpt).....	41
B.3 Environment Parameters — infra/main.parameters.prod.json.....	42
B.4 Deploy Commands.....	42

1. Introduction

1.1 Purpose

This Technical Architecture Document (TAD) defines the end-to-end technical architecture of the Docaya Electronic Document Management System (EDMS). It is the authoritative engineering reference that translates the functional and business requirements into a concrete, cloud-native solution built entirely on Microsoft Azure, Microsoft 365, SharePoint Online and Microsoft Entra ID. It describes the solution architecture, the application (component) architecture, the supporting data, integration, security, infrastructure, network and operations views, and — from version 1.1 — the development methodology, package structure and packaged deployment approach used to build and deliver the system to a client environment.

1.2 Scope

In scope	Out of scope
Logical and physical architecture; application components and modules; data model and stores; integration with SharePoint and AI services; identity and access; security, network and encryption; development methodology and repository/package structure; deployment, observability and operations; non-functional design; and architecture decisions.	Detailed low-level class design and source code; project plans and commercials; organisation-specific naming and data; and migration of pre-existing content, which is handled by a separate workstream.

1.3 Intended Audience

- **Solution & technical architects** — to validate architecture fitness and evolve the design.
- **Engineering teams** — to implement components against a shared blueprint and repository standard.
- **Security, identity and compliance reviewers** — to assess controls, network posture and residency.
- **Platform and operations teams** — to provision, deploy, monitor and run the system in the client environment.

1.4 Definitions & Acronyms

Term	Meaning
EDMS	Electronic Document Management System
Staging Zone	Controlled holding area (Blob or SharePoint staging) before approval
Promotion	Moving an approved document from staging into the live SharePoint library
SoR	System of Record (SharePoint Online for content; Azure SQL for metadata/audit)
RAG	Retrieval-Augmented Generation (grounded AI search)
RBAC / ABAC	Role- / Attribute-Based Access Control
BFF	Backend-for-Frontend
OBO	On-Behalf-Of OAuth flow
MI	Managed Identity (secretless workload identity)
APIM	Azure API Management
IaC	Infrastructure as Code (Bicep)
CI/CD	Continuous Integration / Continuous Delivery
RPO / RTO	Recovery Point / Time Objective

1.5 Requirement Conventions

Normative statements use RFC-2119 keywords: MUST, SHOULD and MAY. Architectural decisions that constrain implementation are captured as Architecture Decision Records (ADRs) in Section 22.

2. Architecture Goals & Principles

2.1 Architecture Goals

#	Goal	Rationale
G1	Governed publishing	No content becomes a record until registered, classified and approved.
G2	Single source of truth	Content in SharePoint Online; metadata, workflow and audit in Azure SQL.
G3	Secure by default	Zero-trust, least privilege, everything authenticated and audited.
G4	Cloud-native & elastic	Managed PaaS services that scale horizontally and fail over cleanly.
G5	Event-driven & decoupled	Long-running and side-effecting work is asynchronous and idempotent.
G6	Intelligent retrieval	Permission-trimmed hybrid search and grounded AI answers over approved content.
G7	Operable & observable	End-to-end tracing, actionable alerts and repeatable deployments.
G8	Repeatably deployable	One packaged release provisions and deploys the whole platform to any environment.

2.2 Guiding Principles

Principle	Design implication
SharePoint is the content system of record	Published binaries and native versions live in governed libraries; no lock-in.
Staging-first	Incomplete or unapproved content is isolated in a controlled staging zone.
Configuration over code	Document types, routing, retention and notifications are managed, not hard-coded.
Least privilege everywhere	Human and workload identities receive only the access they need, at resource scope.
Idempotent side effects	Promotion, indexing and notifications tolerate retries without duplication.
Defence in depth	Controls are layered across identity, network, application and data.
Everything as code	Infrastructure, configuration and pipelines are versioned and reproducible.

2.3 Architecturally Significant Quality Attributes

Attribute	Target / approach
Security	Zero-trust, deny-by-default RBAC+ABAC, private networking, encryption, immutable audit.
Availability	99.9% baseline via zone-redundant PaaS and health-based routing.
Performance	p95 page load < 1.5s; search p95 < 400 ms.
Scalability	Stateless services scale horizontally; async workers absorb bursts.
Reliability	Idempotent operations; retries with backoff; dead-letter handling.
Recoverability	RPO ≤ 5 min, RTO ≤ 1 hr; geo-redundant backups.
Maintainability	Modular services, clean adapters, IaC and automated pipelines.
Deployability	Immutable images, versioned IaC, gated CI/CD, one-command environment provisioning.
Accessibility	WCAG 2.2 AA; bilingual and right-to-left capable presentation.

2.4 Constraints & Assumptions

- The platform is delivered exclusively with native Azure and Microsoft 365 services; no third-party platform is required.
- A licensed SharePoint Online tenant and a federated national single-sign-on provider are available in the client tenant.
- All data resides within the approved sovereign Azure region(s) to meet residency requirements.
- Identity is centralised: user access is governed inside the application, while content operations use a governed service identity.

3. Architecture Approach & Views

The architecture is described through complementary views so that each stakeholder can reason about the parts that matter to them. Together they provide a complete and consistent picture of the system, from the running services to the way the software is built and deployed.

View	Question it answers	Where in this document
Solution / physical	Which services run and how do they connect?	Section 4
Application / logical	How is the software organised into components?	Section 5
Process / runtime	How does work flow end to end?	Section 6
Data	What is stored, where, and under what rules?	Section 7
Integration	How do systems and identities interoperate?	Section 8
Security	How is the system protected?	Section 9
Infrastructure / deployment	How is it provisioned and released?	Sections 10, 20
Observability / operations	How is it observed and run?	Section 11
Development / build	How is the software developed and packaged?	Sections 18, 19
Network	How is traffic segmented and secured?	Section 21
Packaged delivery	How is it deployed at the client site?	Section 22

Reading the two core diagrams Section 4 presents the Solution Architecture — the connected view of Azure and Microsoft 365 building blocks and the data flows between them. Section 5 presents the Application Architecture — the internal, layered composition of the software. Both are shown full-size on the following landscape pages and then described in detail.

4. Solution Architecture

The solution architecture organises the platform into eight cooperating zones — users and channels, identity and single sign-on, the security edge, application services, asynchronous orchestration and messaging, data and content, the Microsoft 365 system of record, and the cross-cutting platform. The diagram overleaf shows every component and the integration paths between them, including the staging-first promotion path and the post-publish retrieval-augmented indexing path. A numbered one-to-ten sequence traces a document from sign-in to searchable record.

The figure is presented full-width on the next (landscape) page for legibility; the layers are then described in detail in Sections 4.1–4.8.

Figure 1 — Connected Solution Architecture

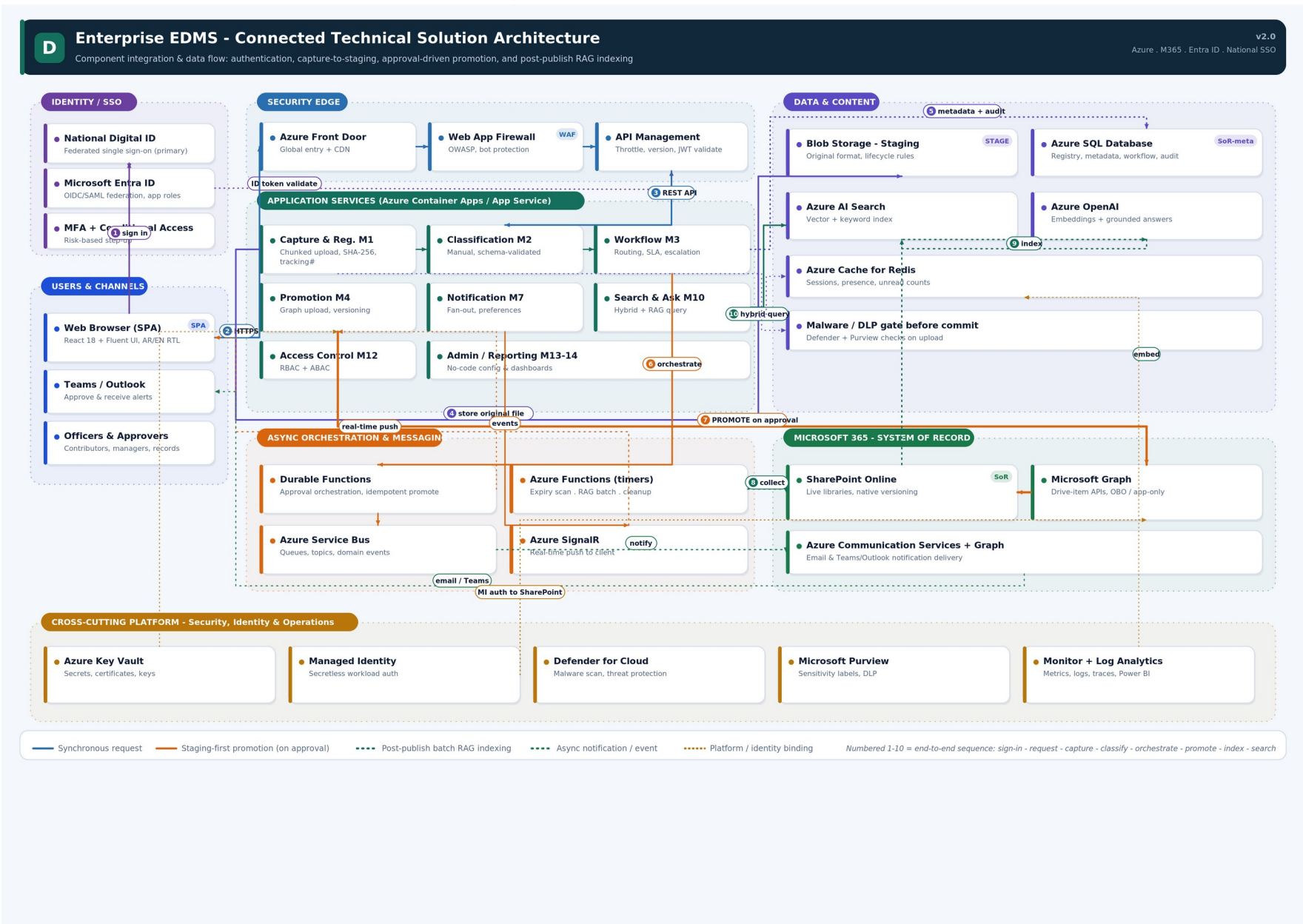


Figure 1 — Solution architecture: Azure & Microsoft 365 building blocks, integration paths, staging-first promotion (orange) and post-publish RAG indexing (teal).

4.1 Users & Channels

Contributors, approvers, records officers and administrators interact through a single-page web application, with Microsoft Teams and Outlook as secondary channels for approvals and notifications. The client is a modern, accessible experience that is bilingual and right-to-left capable.

Component	Responsibility
Web Browser (SPA)	Primary experience for capture, classification, approvals, search and administration.
Teams / Outlook	Approve documents and receive alerts within existing collaboration tools.

4.2 Identity & Single Sign-On

Authentication is federated: a national digital identity provider is the primary sign-on, federated to Microsoft Entra ID which issues application tokens, enforces multi-factor authentication and conditional access, and drives step-up authentication for sensitive actions.

Component	Responsibility
National Digital ID	Primary, standards-based single sign-on for all users.
Microsoft Entra ID	OIDC/SAML federation, application roles, token issuance.
MFA + Conditional Access	Risk-based and step-up authentication policies.

4.3 Security Edge

All ingress passes through a hardened edge. Azure Front Door provides global entry and content delivery, a Web Application Firewall blocks common threats and bots, and Azure API Management enforces throttling, versioning and JSON Web Token validation before any request reaches a service.

Component	Responsibility
Azure Front Door	Global entry point, routing and CDN.
Web Application Firewall	OWASP protection and bot mitigation.
Azure API Management	Rate limiting, versioning, token validation, policy enforcement.

4.4 Application Services

The application tier hosts the domain modules as independently deployable services on Azure Container Apps or App Service. Each service owns a bounded context — capture and registration, classification, workflow and approval, promotion, notification, search, access control, and administration and reporting — and emits domain events for loose coupling and audit.

4.5 Asynchronous Orchestration & Messaging

Long-running and side-effecting work is asynchronous. Durable Functions orchestrate the approval and idempotent promotion process; Azure Service Bus carries queues, topics and domain events; timer-triggered Azure Functions run the expiry scan, the batch indexing job and staging cleanup; and Azure SignalR pushes real-time updates to the client.

Component	Responsibility
Durable Functions	Stateful orchestration of approval and promotion; idempotent by design.
Azure Service Bus	Reliable queues, topics and event distribution.
Azure Functions (timers)	Expiry reminders, batch RAG indexing, staging lifecycle cleanup.
Azure SignalR	Real-time notification push to connected clients.

4.6 Data & Content

The data zone separates transient staging from durable stores. Azure Blob Storage holds the original file during capture, classification and approval; Azure SQL Database is the authoritative registry for metadata, workflow state and the hash-chained audit; Azure AI Search holds the vector and keyword index; Azure OpenAI provides embeddings and grounded answers; and Azure Cache for Redis serves sessions, presence and unread counts. A malware and data-loss-prevention gate validates every upload before it can proceed.

Component	Responsibility
Blob Storage (Staging)	Holds the original file in its native format with lifecycle rules.
Azure SQL Database	Registry, metadata, workflow state and tamper-evident audit.
Azure AI Search	Hybrid vector + keyword index over approved content.
Azure OpenAI	Embeddings and grounded, cited answers for the assistant.
Azure Cache for Redis	Sessions, presence and unread notification counts.
Content-safety gate	Malware scan and DLP/classification checks before commit.

4.7 Microsoft 365 — System of Record

Approved content lives in SharePoint Online, which provides governed libraries and native versioning. Microsoft Graph is the integration surface for drive-item operations, performed under a governed workload identity. Azure Communication Services and Graph deliver email and Teams/Outlook notifications.

Component	Responsibility
SharePoint Online	Live libraries, native versioning; content system of record.
Microsoft Graph	Drive-item APIs for promotion and retrieval; OBO / application permissions.
Communication Services + Graph	Email and Teams/Outlook notification delivery.

4.8 Cross-Cutting Platform

Platform services apply to every zone: Azure Key Vault for secrets, certificates and keys; Managed Identity for secretless workload authentication; Microsoft Defender for Cloud for threat protection and upload malware scanning; Microsoft Purview for sensitivity labelling and data-loss prevention; and Azure Monitor with Log Analytics (and Power BI) for metrics, logs, traces and dashboards.

Staging-first & RAG paths The orange path shows that a document is promoted into the correct SharePoint library only on approval, in its original format. The teal path shows that retrieval-augmented indexing runs as a batch over approved, published content — never during capture or classification.

5. Application Architecture

The application architecture describes how the software itself is composed. It follows a clean, layered structure with clear dependency direction from top to bottom: a presentation tier, an API gateway and backend-for-frontend, the application (domain) services, a domain model and business-logic layer, integration and anti-corruption adapters, and a data-access and persistence layer. Cross-cutting concerns apply to every layer, and asynchronous workers handle orchestration and scheduled processing.

The component diagram is presented full-width on the next (landscape) page and then described tier by tier in Sections 5.1–5.7.

Figure 2 — Detailed Application Architecture

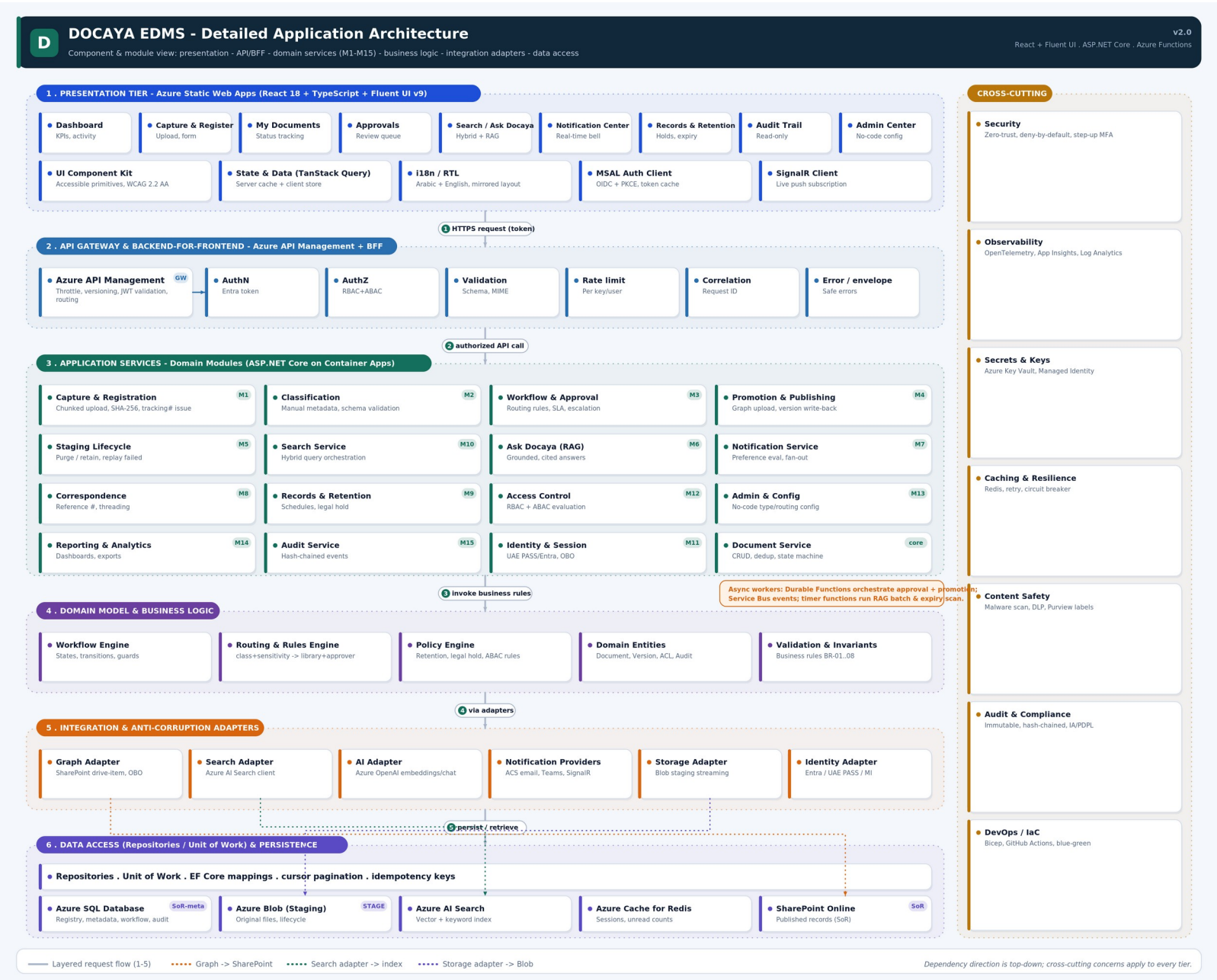


Figure 2 — Application architecture: presentation, API/BFF, application services (M1–M15), domain logic, integration adapters, data access, and cross-cutting concerns.

5.1 Presentation Tier

The presentation tier is a React 18 and TypeScript single-page application built with Fluent UI and hosted on Azure Static Web Apps. It is composed of nine feature modules and five foundation blocks.

Group	Components
Feature modules	Dashboard · Capture & Register · My Documents · Approvals · Search / Ask · Notification Center · Records & Retention · Audit Trail · Admin Center
Foundation	Accessible UI component kit · State & data (server cache + client store) · i18n / RTL · Auth client (OIDC + PKCE) · Real-time client (SignalR)

5.2 API Gateway & Backend-for-Frontend

A backend-for-frontend behind Azure API Management aggregates reads, shapes responses for the client, and hosts the request-processing pipeline. Every request passes through authentication, authorization (RBAC + ABAC), input validation, rate limiting, correlation-ID assignment and a normalised error envelope.

Pipeline stage	Function
Authentication	Validate the Entra-issued bearer token.
Authorization	Enforce deny-by-default RBAC + ABAC for the operation and object.
Validation	Validate body, schema, MIME type and identifiers.
Rate limiting	Protect services per key/user.
Correlation	Attach an end-to-end request ID.
Error handling	Return safe, structured error envelopes without leaking internals.

5.3 Application Services (Domain Modules)

The domain modules implement the business capabilities. Each is a cohesive service with a single responsibility, communicating synchronously via the BFF and asynchronously via events.

Module	Responsibility
M1 Capture & Registration	Chunked upload, SHA-256 hashing, tracking-number issue, registry creation.
M2 Classification	Manual, schema-validated metadata capture.
M3 Workflow & Approval	Routing rules, SLA timers, escalation, decision handling.
M4 Promotion & Publishing	Graph upload of the original file, version write-back.
M5 Staging Lifecycle	Purge or retain staging, replay failed promotions.
M6 Ask (RAG)	Grounded, cited answers over permitted content.
M7 Notification	Preference evaluation and multi-channel fan-out.
M8 Correspondence	Reference numbers, threading and action tracking.
M9 Records & Retention	Retention schedules, disposition and legal hold.
M10 Search	Hybrid keyword + semantic query orchestration.
M11 Identity & Session	Federated sign-on, OBO and session management.
M12 Access Control	RBAC + ABAC effective-permission evaluation.
M13/M14 Admin & Reporting	No-code configuration, dashboards and exports.
M15 Audit	Hash-chained, tamper-evident event recording.
Document Service (core)	CRUD, deduplication and the document state machine.

5.4 Domain Model & Business Logic

Business rules are centralised in a domain layer independent of infrastructure. It contains the workflow engine (states, transitions and guards), the routing and rules engine (mapping class and sensitivity to library and approvers), the policy engine (retention, legal hold and ABAC rules), the domain entities (document, version, ACL, audit), and the invariants that enforce the platform's business rules.

5.5 Integration & Anti-Corruption Adapters

External systems are reached only through adapters that isolate the domain from provider-specific detail. This keeps the core clean and makes providers replaceable.

Adapter	Encapsulates
Graph Adapter	SharePoint drive-item operations via Microsoft Graph (OBO / app-only).
Search Adapter	Azure AI Search indexing and query.
AI Adapter	Azure OpenAI embeddings and chat completions.
Notification Providers	Communication Services email, Teams and SignalR.
Storage Adapter	Streaming to and from Blob staging.
Identity Adapter	Entra, national SSO and Managed Identity token acquisition.

5.6 Data Access & Persistence

A repository and unit-of-work layer with EF Core mappings mediates all persistence, providing cursor pagination and idempotency keys. It fronts Azure SQL (registry, metadata, workflow, audit), Blob staging (original files), Azure AI Search (index), Redis (cache) and SharePoint Online (published records).

5.7 Cross-Cutting Concerns & Async Workers

Security, observability, secrets management, caching and resilience, content safety, audit and DevOps apply across all layers. Asynchronous workers — Durable Functions, Service Bus consumers and timer functions — orchestrate approval and promotion, distribute events, and run the batch indexing and expiry jobs, all with idempotent, retry-safe semantics.

6. Process & Runtime View — End-to-End Lifecycle

The platform enforces a controlled capture → classify → approve → promote pipeline, followed by batch indexing and continuous access governance. The table below is the authoritative runtime sequence.

#	Stage	Runtime behaviour	Status
1	Capture / Register	Original file streamed to Blob staging; SHA-256 computed; tracking number issued; registry row created in Azure SQL.	PendingRegistration
2	Classify	Contributor selects class, sensitivity, department and approval flow; metadata validated and persisted.	Classified
3	Submit	Routing rules resolve target library and approver(s); item enqueued to the approver.	PendingApproval
4	Notify	Events raised to the Notification Service; in-app, email and Teams alerts delivered.	(no change)
5	Decide	Approver approves, or rejects with mandatory comments returning the item to the initiator.	Approved / Returned
6	Promote	Durable Functions call Graph to upload the original file to the correct library; item ID, URL and version 1.0 written back.	Published
7	Cleanup	Staging reconciled (purge or retain); contributor notified of publication.	Reconciled
8	Index (RAG)	Batch job chunks and embeds published content and metadata into Azure AI Search.	Indexed
9	Access & Govern	Federated SSO; RBAC+ABAC evaluation; audit; retention and legal hold applied.	Continuous

6.1 Consistency & Idempotency

- Azure SQL is authoritative for metadata, workflow state and audit; SharePoint is authoritative for content.
- Promotion carries an idempotency key so a retry can never create a duplicate SharePoint item.
- SharePoint item ID, URL and version are committed only after a successful upload; failures remain recoverable in staging.
- Indexing is asynchronous, permission-aware and fully re-runnable.

7. Data Architecture

7.1 Storage Strategy

Store	Purpose	Notes
Azure SQL Database	Registry, metadata, workflow state, audit	Authoritative for all structured data; hash-chained audit.
Azure Blob (Staging)	Original files pre-approval	Native format preserved; lifecycle rules for cleanup.
SharePoint Online	Published records	Content system of record with native versioning.
Azure AI Search	Vector + keyword index	Built from approved content; permission-trimming keys.
Azure Cache for Redis	Sessions, presence, counts	Low-latency ephemeral state.
Archive tier / repository	Cold records	Retention-driven archival with registry linkage.

7.2 Core Entities

Entity	Purpose	Key fields
Document	Central registry & lifecycle	TrackingNumber, Status, Class, Sensitivity, ExpiryDate, SharePointItemId
Version	Immutable version reference	VersionLabel, SharePoint version, CreatedAt
AccessEntry	ABAC / scoping	PrincipalId, PrincipalType, Permission
WorkflowInstance	Approval state & history	Stage, AssignedTo, Decision, ActedAt
Notification	Durable notification history	UserId, Type, Channel, ReadAt
AuditEvent	Tamper-evident audit	ActorId, Action, RequestId, PrevHash, Hash
RetentionPolicy	Lifecycle rules	Trigger, Duration, Disposition

7.3 Data Classification & Handling

- Every document carries a sensitivity marking (Public, Internal, Confidential, Secret) that drives access and routing.
- Sensitivity labels and data-loss-prevention policies are applied through Microsoft Purview.
- Personal data is minimised, access-controlled and processed lawfully in line with applicable data-protection law.

7.4 Retention, Legal Hold & Residency

- Time- or event-based retention schedules govern disposition (auto-archive, review-then-delete, or approved deletion).
- Legal hold overrides retention, freezes documents and blocks deletion, fully audited.
- All data resides within the approved sovereign Azure region(s).

8. Integration Architecture

8.1 SharePoint & Microsoft Graph

The platform integrates with SharePoint Online through Microsoft Graph. A single governed service identity holds the required read/write scope over the document estate; user access is decided inside the application before any Graph call is made. Promotion uploads the original file to the resolved library; native versioning assigns the version, which is recorded against the registry.

Aspect	Design decision	Benefit
User access	Governed inside the application (RBAC + ABAC)	Single, consistent control point
Content operations	Executed by the governed service identity	Uniform, auditable access to content
Credential	Managed Identity / workload identity	Secretless; least privilege
Attribution	Actions attributed to the real user in Azure SQL	Full accountability

8.2 AI & Retrieval-Augmented Generation

A batch pipeline collects newly published content, extracts and chunks text, generates embeddings with Azure OpenAI, and upserts vectors, metadata and permission-trimming keys into Azure AI Search. Queries fuse keyword and vector relevance; the assistant answers only from content the user may access and cites its sources. AI is never invoked during capture or classification.

8.3 Notifications

Domain events flow through Service Bus to the Notification Service, which evaluates per-user preferences and quiet hours and fans out across in-app (SignalR), email (Communication Services) and Teams/Outlook (Graph). Security notifications are mandatory. Delivery is de-duplicated and idempotent.

8.4 API & Contracts

- Public contracts are versioned under a stable path and described with an OpenAPI document validated in the pipeline.
- Writes are idempotent; collection endpoints use cursor pagination with deterministic ordering.
- A consistent error envelope is returned for validation, authorization, not-found, conflict, throttling and dependency failures.

9. Security Architecture

9.1 Identity & Access

- **Authentication:** federated national SSO to Entra ID; OIDC + PKCE for the SPA; MFA and conditional access enforced.
- **Authorization:** deny-by-default RBAC plus ABAC on department, sensitivity and clearance; SharePoint ACLs remain decisive for content.
- **Workload identity:** Managed Identity for service-to-service and SharePoint access; no stored user passwords.
- **Step-up:** additional verification for sensitive actions such as approving Secret documents or changing permissions.

9.2 Network & Perimeter

Control	Design
Ingress	Front Door + WAF + API Management; only published, authenticated endpoints are exposed.
Private connectivity	Private endpoints for SQL, Blob and Key Vault; disabled public access where feasible.
Egress & DNS	Controlled egress and private DNS resolution from the application to data services.
Segmentation	Least-privilege network paths between tiers; no direct client access to data stores.

9.3 Data Protection & Content Safety

- Encryption in transit (TLS) and at rest (platform-managed keys; customer-managed keys optional by policy).
- Every upload is quarantined and scanned for malware; DLP and sensitivity checks run before commit.
- Secrets, certificates and keys are held in Key Vault and never exposed in logs or configuration exports.

9.4 Audit & Tamper-Evidence

Every action — register, classify, submit, decide, promote, view, download and permission change — is recorded in a hash-chained audit log in Azure SQL, capturing actor, action, object, tracking number, timestamp, request ID, source and outcome. The chain makes tampering evident, and records can be exported to immutable storage for legal purposes.

9.5 Threat Model (summary)

Threat	Primary mitigation
Stolen credential / token replay	Managed Identity, short-lived tokens, audience validation, anomaly alerts.
Over-privileged access	Resource-scoped least privilege; periodic access review.
Malicious upload	Quarantine, malware scan, MIME/signature validation, DLP.
Unauthorized search result	Permission-trimmed indexing and re-check at open/download.
Duplicate publication on retry	Idempotency keys and item reconciliation.
Data exfiltration	Private networking, DLP, audit and alerting.

10. Infrastructure & Deployment (Overview)

This section gives the high-level hosting and release model. The detailed infrastructure layer, Infrastructure-as-Code module structure and provisioning order are specified in Section 20; the network design in Section 21; and the packaged client-site deployment in Section 22.

10.1 Compute & Hosting

Concern	Choice	Rationale
Frontend hosting	Azure Static Web Apps	Global delivery, integrated auth and CI/CD.
API & services	Azure Container Apps / App Service	Elastic, containerised, managed scaling.
Async workers	Azure Functions / Durable Functions	Event- and timer-driven orchestration.
Gateway	Azure API Management	Policy, throttling, versioning.

10.2 Environments

Isolated development, test, staging and production environments are provisioned identically through Infrastructure-as-Code (Bicep), with environment-specific configuration and immutable application images. Production credentials and role assignments are created by an authorised deployment identity, separate from developer accounts.

10.3 CI/CD & Release Gates

- Build, type-check, lint and dependency audit.
- Unit, integration, end-to-end, accessibility and API-contract tests.
- Container, IaC, secret, SAST and DAST security scans.
- Infrastructure validation and reviewed what-if under an authorised identity.
- Private-endpoint and DNS connectivity verified from the running workload.
- Database migration and rollback rehearsal; synthetic health checks; blue-green or canary release.

11. Observability & Operations

11.1 Telemetry

The application is instrumented with OpenTelemetry; traces, metrics and logs flow to Application Insights and Log Analytics, correlated by the end-to-end request ID so a document's journey from capture to publication can be reconstructed. Power BI surfaces operational and compliance dashboards.

11.2 Monitoring & Alerting

Signal	Alert condition	Response
Token / auth failures	Sustained 401/403 or issuer mismatch	Validate identity, claims and scope.
AI dependency	Throttling or latency breach	Backoff, inspect quota, degrade gracefully.
Promotion backlog	Queue age or dead-letter threshold	Reprocess safely via idempotent retry.
Search drift	Reconciliation mismatch	Re-index and correct ACL source.
Expiry job failure	Missed run or delivery spike	Run recovery job; confirm owner notification.

11.3 SLOs & Runbooks

- Service-level objectives with error budgets for availability and latency.
- Documented runbooks for sign-in, capture, approval, promotion, search and audit journeys.
- Synthetic monitoring of critical flows and on-call escalation.

12. Non-Functional Requirements

Attribute	Target	Verification
Availability	99.9% baseline	Synthetic checks; monthly SLO report.
Page load	p95 < 1.5 s	Browser telemetry; load tests.
Search latency	p95 < 400 ms	Permission-trimmed query tests.
File size	Up to 5 GB (resumable)	Interruption, retry and memory tests.
Recovery	RPO ≤ 5 min · RTO ≤ 1 hr	Restore and regional recovery exercise.
Accessibility	WCAG 2.2 AA; RTL	Automated and manual audits.
Scalability	10k+ concurrent; 1B+ documents	Load and soak testing.
Integration reliability	Idempotent, bounded retry	Contract and failure-injection tests.

13. Resilience, Availability & Disaster Recovery

- Zone-redundant PaaS and stateless services enable horizontal scale and graceful failover.
- Asynchronous workers absorb bursts; dead-letter queues capture and replay poison messages.
- Geo-redundant backups with continuous and daily snapshots; documented restore procedures.
- Retries with exponential backoff and circuit breakers protect against transient dependency failures.
- Failed promotions and indexing are recoverable and idempotently re-runnable.

14. Compliance & Governance

- Designed to align with recognised information-assurance standards and applicable data-protection law.
- Data residency pinned to approved sovereign region(s).
- Immutable, hash-chained audit supports evidential integrity and defensible disposition.
- Right-to-be-forgotten and data-subject processes supported through governed workflows.

15. Technology Stack Summary

Concern	Technology
Frontend	React 18 + TypeScript + Fluent UI on Azure Static Web Apps
Gateway / BFF	Azure API Management + backend-for-frontend
Services	ASP.NET Core on Azure Container Apps / App Service
Async / workflow	Azure Functions, Durable Functions, Azure Service Bus
Real-time	Azure SignalR Service
Content store	SharePoint Online via Microsoft Graph
Staging	Azure Blob Storage
Metadata / audit	Azure SQL Database
Search / AI	Azure AI Search + Azure OpenAI
Cache	Azure Cache for Redis
Secrets	Azure Key Vault + Managed Identity
Security	Defender for Cloud, Microsoft Purview
Observability	OpenTelemetry, Application Insights, Log Analytics, Power BI
IaC / CI-CD	Bicep + pipeline automation (GitHub Actions / Azure DevOps)

16. Architecture Decision Records

ID	Decision	Status
ADR-01	SharePoint Online is the content system of record; Azure SQL is authoritative for metadata and audit.	Accepted
ADR-02	All content follows staging → classify → approve → promote; nothing publishes ungoverned.	Accepted
ADR-03	User access is governed in-application; content operations use one governed service identity.	Accepted
ADR-04	AI/RAG runs as a batch over approved content only; never at capture or classification.	Accepted
ADR-05	Asynchronous orchestration via Durable Functions and Service Bus with idempotent operations.	Accepted
ADR-06	Zero-trust with private networking, Managed Identity and deny-by-default authorization.	Accepted
ADR-07	Delivered exclusively on native Azure and Microsoft 365 services.	Accepted
ADR-08	UI, API, workers and infrastructure are separately packaged and independently deployable.	Accepted

17. Risks & Mitigations

Risk	Impact	Mitigation
Graph API throttling during bulk promotion	Delayed publishing	Batching, retry with backoff, queue smoothing.
Staging growth	Cost	Lifecycle cleanup after promotion.
Complex ABAC rules	Misconfiguration	Rule simulation and review before publish.
Permission drift in SharePoint	Access errors	Service identity + periodic reconciliation.
Index staleness	Stale search results	Scheduled batch cadence and full re-index capability.

18. Development Methodology & Guidelines (VS Code)

This section defines how the platform is engineered day-to-day using Visual Studio Code (VS Code). It establishes the workspace model, the toolchain, coding standards, branching and review process, and the local run/debug loop for each independently developed package — the web UI, the API, the asynchronous workers/workflow, and the shared contracts. Each package is developed, versioned, tested and packaged separately, and can be run in isolation on a developer machine.

18.1 Delivery Approach

Delivery is iterative and trunk-based. Work is planned as small, vertically-sliced increments (a thin slice through UI → API → data), each protected by automated tests and a definition of done. Short-lived feature branches are merged into the main branch through pull requests that must pass all quality gates. Every merge to main produces versioned, immutable build artifacts that are promoted unchanged across environments.

Practice	Standard
Methodology	Agile increments with trunk-based development and continuous integration.
Branching	main (always releasable) + short-lived feature/*, fix/*, chore/* branches.
Merge policy	Pull request with review, green pipeline, and up-to-date branch; squash merge.
Versioning	Semantic Versioning (MAJOR.MINOR.PATCH); Git tag per release.
Commits	Conventional Commits (feat, fix, chore, docs, refactor, test).
Definition of Done	Code + tests + accessibility + security checks + docs + telemetry emitted.

18.2 Separation into Independently Developed Packages

The solution is deliberately separated so that the presentation, the API, the asynchronous processing and the shared contracts are developed, built and released as distinct packages. This lets teams work in parallel, keeps build times short, allows each part to scale and deploy on its own cadence, and produces clean, independently versioned artifacts.

Package	What it is	Built with	Produced artifact
Web UI (SPA)	React 18 + TypeScript + Fluent UI single-page app	Vite	Static bundle (Static Web App)
API / BFF	ASP.NET Core Web API and backend-for-frontend	dotnet + Docker	Container image
Workers / Workflow	Azure Functions + Durable Functions	dotnet + Functions	Function app package (zip)
Shared Contracts	OpenAPI spec + generated TypeScript & C# clients, DTOs	OpenAPI generator	Versioned client packages
Infrastructure	Bicep modules + parameters	Bicep	ARM template + parameter files

Why separate packages Separating the UI, API, workers and infrastructure gives independent build/test/release cycles, smaller and faster pipelines, clear ownership boundaries, and the ability to roll a single package forward or back without touching the others — while a shared, versioned contract keeps the UI and API in lock-step.

18.3 Developer Workstation Prerequisites

Every developer prepares an identical local toolchain. Versions are pinned so that all machines and the CI runners build the software the same way.

Tool	Purpose	Notes
Visual Studio Code	Primary IDE for all packages	With the workspace extensions below
Node.js LTS + npm	Build & run the Web UI	Version pinned in .nvmrc / engines
.NET SDK (LTS)	Build & run the API and Functions	Version pinned in global.json
Azure Functions Core Tools	Run/debug Functions locally	Durable Functions emulation
Docker Desktop	Build & run API container images	Local container parity with cloud
Azure CLI + Bicep CLI	Provision and validate infrastructure	az bicep build / what-if
Git	Version control	Conventional Commits enforced by hook

18.4 VS Code Workspace & Recommended Extensions

The repository ships a multi-root VS Code workspace so a single window opens all packages with the right settings, tasks and debug profiles. Recommended extensions are declared in `.vscode/extensions.json` so VS Code prompts each developer to install the exact set.

18.4.1 Multi-root workspace (`docaya.code-workspace`)

```
{
  "folders": [
    { "name": "web", "path": "src/web" },
    { "name": "api", "path": "src/api" },
    { "name": "functions", "path": "src/functions" },
    { "name": "contracts", "path": "src/contracts" },
    { "name": "infra", "path": "infra" }
  ],
  "settings": {
    "editor.formatOnSave": true,
    "editor.codeActionsOnSave": { "source.fixAll": "explicit" },
    "dotnet.defaultSolution": "src/api/Docaya.Api.sln"
  }
}
```

18.4.2 Recommended extensions (`.vscode/extensions.json`)

Extension	Used for
ESLint · Prettier	Lint and format the Web UI (TypeScript/React)
C# Dev Kit	Build, run and debug the ASP.NET Core API
Azure Functions	Run, debug and publish the Functions app
Azure Resources · Bicep	Author, validate and deploy Infrastructure-as-Code
REST Client / OpenAPI (Swagger)	Explore and test the API contract
Docker	Build and inspect container images
GitLens	Review history and authorship during code review
axe Accessibility Linter	Catch accessibility issues while coding the UI

18.5 Getting Started — First Run in VS Code

A new engineer clones the repository, opens the workspace, and restores each package. The commands below are also wired as VS Code tasks (Terminal → Run Task) so they can be launched from the command palette.

```
# 1) clone and open the multi-root workspace
git clone <repo-url> docaya && cd docaya
code docaya.code-workspace

# 2) Web UI (folder: src/web)
npm ci # install exact dependencies from lockfile
npm run dev # start Vite dev server on http://localhost:5173

# 3) API (folder: src/api)
dotnet restore && dotnet run # https://localhost:5001 (Swagger UI at /swagger)

# 4) Functions / workers (folder: src/functions)
func start # runs Durable + timer functions locally

# 5) Infrastructure (folder: infra)
az bicep build --file main.bicep # compile & validate templates
```

18.6 Local Debug Profiles (`.vscode/launch.json`)

Each package has a first-class debug profile, and a compound profile launches the UI, API and Functions together so a full flow can be stepped through locally. Breakpoints, watch windows and hot-reload work in every package.

Debug profile	Launches	Attach / reload
Web: Vite + Chrome	SPA with source maps in Chrome	Hot Module Reload
API: .NET	ASP.NET Core API with Swagger	Hot reload on save

Debug profile	Launches	Attach / reload
Functions: Host	Durable + timer functions	Local storage emulator
Compound: Full stack	UI + API + Functions together	Single F5 to run all

18.7 Coding Standards & Quality Gates

- **Formatting & linting:** Prettier + ESLint for the UI; dotnet format + analyzers for the API; enforced on save and in CI.
- **Static analysis:** SAST on every pull request; nullable reference types and treat-warnings-as-errors for .NET; strict TypeScript.
- **Testing:** unit tests (Vitest/xUnit), integration tests against an isolated database and mocked Graph, end-to-end tests (Playwright) and accessibility tests (axe).
- **Coverage:** minimum coverage threshold for core logic enforced in the pipeline.
- **Secrets:** no secrets in code or config; local development uses user-secrets / .env excluded by .gitignore; cloud uses Key Vault + Managed Identity.
- **Pre-commit hooks:** format, lint and commit-message checks run before a commit is accepted.

18.8 Contract-First Development

The API contract is authored as an OpenAPI 3.1 document in the shared contracts package. Typed clients are generated from it: a TypeScript client for the UI and a C# client for internal callers. This guarantees the UI and API never drift, and contract tests in CI fail the build if a breaking change is introduced without a version bump.

```
# regenerate typed clients from the OpenAPI contract
npm run gen:client    # -> src/web/src/api (TypeScript)
dotnet tool run nswag # -> src/api/Generated (C#)
```

19. Repository & Package Structure

The codebase is a single repository (mono-repo) with clearly separated, independently buildable packages. This gives one place for versioning, cross-cutting changes and pull requests, while preserving strong boundaries and independent artifacts for the UI, API, workers, contracts and infrastructure.

19.1 Top-Level Layout

```
docaya/
├── .github/workflows/    # CI, security, and release pipelines
├── .vscode/              # workspace tasks, launch, extensions
├── docs/                 # ADRs, API reference, runbooks, threat model
├── infra/                # Bicep IaC (see Section 20)
│   ├── main.bicep
│   ├── main.parameters.<env>.json
│   └── modules/          # one module per resource group of services
├── src/
│   ├── web/              # React SPA package
│   ├── api/              # ASP.NET Core API + BFF package
│   ├── functions/        # Azure Functions / Durable workers package
│   ├── contracts/        # OpenAPI + generated clients + shared DTOs
│   ├── tests/            # unit, integration, e2e, a11y, load
│   ├── deploy/           # packaged-release scripts (Section 22)
│   ├── docaya.code-workspace # multi-root VS Code workspace
│   └── README.md
└──
```

19.2 Web UI Package (src/web)

```
src/web/
├── src/
│   ├── app/              # shell, routing, providers, startup
│   ├── features/         # dashboard, capture, approvals, search, admin ...
│   ├── components/       # accessible Fluent UI primitives
│   ├── services/         # typed API client (generated from contract)
│   ├── hooks/ i18n/ styles/ types/
│   ├── public/           # static assets, localisation catalogs (ar, en)
│   ├── index.html vite.config.ts tsconfig.json
│   └── package.json      # scripts: dev, build, test, lint, gen:client
└──
```

19.3 API / BFF Package (src/api)

```
src/api/
├── Docaya.Api/            # HTTP host, controllers, middleware pipeline
├── Docaya.Application/    # use-cases, DTOs, validators
├── Docaya.Domain/         # entities, workflow/rules/policy engines
├── Docaya.Infrastructure/ # EF Core, Graph/Search/AI/Storage adapters
├── Docaya.Api.sln
└── Dockerfile            # multi-stage build -> immutable image
```

19.4 Workers / Workflow Package (src/functions)

```
src/functions/
├── Orchestrations/        # Durable: approval + idempotent promotion
├── Triggers/              # timer: expiry scan, RAG batch, staging cleanup
├── Consumers/             # Service Bus event handlers (notifications)
├── host.json local.settings.json (git-ignored)
└── Docaya.Functions.csproj
```

19.5 Package Boundaries & Dependency Rules

Rule	Statement
Dependency direction	Domain depends on nothing; Application depends on Domain; Infrastructure implements interfaces; API composes them.
No provider leakage	Graph/Search/AI/Storage SDKs appear only inside adapters, never in Domain or Application.
UI ↔ API only via contract	The UI calls the API exclusively through the generated, versioned client.
Shared types	Cross-package types live in the contracts package; no copy-paste DTOs.
Independent versioning	Each package carries its own version and changelog; released on its own cadence.

20. Infrastructure Layer — IaC & Provisioning (Detailed)

All cloud resources are defined as code with Bicep and provisioned identically across environments. Nothing is created by hand in a portal for shared or production environments; every resource, role assignment, private endpoint and configuration value is versioned, reviewed and reproducible.

20.1 Bicep Module Structure (infra/)

```
infra/
├─ main.bicep           # subscription/RG composition root
├─ main.parameters.dev.json
├─ main.parameters.staging.json
├─ main.parameters.prod.json
├─ modules/
│   ├─ network.bicep    # VNet, subnets, NSGs, private DNS
│   ├─ identity.bicep   # managed identities + role assignments
│   ├─ keyvault.bicep   # Key Vault + private endpoint
│   ├─ sql.bicep        # Azure SQL + private endpoint
│   ├─ storage.bicep    # Blob staging + lifecycle + PE
│   ├─ search.bicep     # Azure AI Search
│   ├─ openai.bicep     # Azure OpenAI + deployments
│   ├─ servicebus.bicep # queues, topics, subscriptions
│   ├─ redis.bicep      # Azure Cache for Redis
│   ├─ containerapp.bicep # API + workers environment
│   ├─ staticwebapp.bicep # Web UI hosting
│   ├─ apim.bicep       # API Management + policies
│   ├─ frontdoor.bicep  # Front Door + WAF
│   └─ observability.bicep # Log Analytics, App Insights, alerts
```

20.2 Resource Inventory (per environment)

Layer	Azure resource	Key configuration
Edge	Front Door + WAF	OWASP ruleset, custom domain, TLS, routing
Gateway	API Management	Products, JWT validation, rate limits, versions
Frontend	Static Web Apps	Global CDN, linked to Entra auth
Compute	Container Apps environment	API + workers, scale rules, managed identity
Async	Functions (plan)	Durable + timer triggers
Messaging	Service Bus	Queues, topics, dead-letter
Registry / Metadata	Azure SQL Database	Private endpoint, TDE, geo-backup
Staging	Blob Storage	Private endpoint, lifecycle rules
Search / AI	AI Search + OpenAI	Private access, model deployments
Cache	Redis	Private endpoint, TLS-only
Secrets	Key Vault	RBAC data-plane, private endpoint
Identity	Managed Identities	Resource-scoped role assignments
Security	Defender for Cloud + Purview	Threat protection, DLP, labels
Observability	Log Analytics + App Insights	Diagnostic settings on every resource

20.3 Provisioning Order & Dependencies

Modules are composed so that foundational resources exist before dependants. The composition root enforces this ordering; a single deployment creates the whole environment.

#	Provision	Depends on
1	Network (VNet, subnets, NSGs, private DNS)	—
2	Identity (managed identities)	—
3	Key Vault (+ private endpoint)	Network
4	Data stores: SQL, Blob, Search, OpenAI, Redis	Network, Key Vault
5	Messaging: Service Bus	Network
6	Observability: Log Analytics, App Insights	—

#	Provision	Depends on
7	Compute: Container Apps, Functions	1–6
8	Edge & gateway: APIM, Front Door + WAF, Static Web Apps	7
9	Role assignments (data-plane RBAC)	2 + 3–8

20.4 Identity, Secrets & Least-Privilege Roles

- Each workload runs under its own user-assigned Managed Identity; there are no stored passwords or connection secrets in application settings.
- Data-plane role assignments are resource-scoped and minimal — for example, the API identity receives only the specific Key Vault, Storage, Search and Service Bus roles it uses.
- The deployment identity holds the rights to create role assignments; it is separate from developer and runtime identities.
- Secrets that must exist (for example third-party keys) are stored in Key Vault and referenced by URI; they are never emitted to logs or ARM outputs.

20.5 Validate & Deploy Commands

```
# compile & preview changes (no mutation)
az bicep build --file infra/main.bicep
az deployment sub what-if \
  --location <region> \
  --template-file infra/main.bicep \
  --parameters infra/main.parameters.prod.json

# provision the environment (authorised deployment identity)
az deployment sub create \
  --location <region> \
  --template-file infra/main.bicep \
  --parameters infra/main.parameters.prod.json
```

20.6 Environment Matrix

Setting	Dev	Staging	Production
High availability	Single zone	Zone-redundant	Zone-redundant (+ geo-backup)
Private endpoints	Optional	Enabled	Enforced
Scale	Minimal	Production-like	Auto-scale to target load
Data	Synthetic	Anonymised	Live (residency-pinned)
Access	Developers	Release + QA	Break-glass + operators only

21. Network Architecture (Detailed)

The network is designed on zero-trust principles: a single, controlled public ingress; private-only connectivity to every data service; segmented subnets with least-privilege paths; and private DNS so that all internal name resolution stays inside the virtual network. No client ever reaches a data store directly.

21.1 Topology

A hub-and-spoke virtual network hosts the platform. Public traffic enters only through Front Door and the Web Application Firewall, is inspected and routed to API Management, and only then reaches the application subnet. Data services are reachable exclusively over private endpoints on a dedicated subnet.

Subnet	Hosts	Inbound from	Outbound to
snet-edge	APIM (internal mode)	Front Door / WAF	App subnet
snet-app	Container Apps (API + workers)	APIM only	Private-endpoint subnet, Service Bus
snet-func	Functions (workers)	Service Bus, timers	Private-endpoint subnet, Graph (egress)
snet-pe	Private endpoints: SQL, Blob, KV, Search, Redis, OpenAI	App/Func subnets	—
snet-mgmt	Deployment / jump connectivity	Authorised operators	Control plane

21.2 Private Connectivity & DNS

- Every data service (SQL, Blob, Key Vault, AI Search, Redis, OpenAI, Service Bus) is exposed only through a Private Endpoint in snet-pe; public network access is disabled.
- Private DNS zones (for example privatelink.database.windows.net, privatelink.blob.core.windows.net, privatelink.vaultcore.azure.net) resolve service names to private IPs inside the VNet.
- The application resolves and connects to data services entirely over the Microsoft backbone; no data-plane traffic traverses the public internet.

21.3 Ingress & Egress Controls

Direction	Control	Design
Ingress	Front Door + WAF	Single global entry; OWASP rules; bot and rate protection; TLS termination.
Ingress	API Management	JWT validation, product keys, throttling before any service is reached.
East-West	NSGs + subnet rules	Least-privilege paths; app cannot be reached except via APIM.
Egress	Controlled outbound	Restricted egress; only required endpoints (e.g., Microsoft Graph) are allowed.
DNS	Private DNS zones	Internal resolution stays within the VNet.

21.4 Traffic Flow (request to data)

Ho p	From → To	Protocol / control
1	User → Front Door/WAF	HTTPS; WAF inspection
2	Front Door → API Management	HTTPS; routing + JWT validation
3	APIM → API (snet-app)	Private; mTLS-capable; RBAC+ABAC in app
4	API → data services (snet-pe)	Private endpoint; Managed Identity
5	API → SharePoint via Graph	HTTPS egress; governed service identity
6	Workers ↔ Service Bus	Private; managed identity

21.5 Network Security Controls

- Network Security Groups on every subnet enforce least-privilege inbound/outbound rules; default-deny with explicit allows.
- No public IPs on compute or data resources; the only public surface is Front Door.
- Optional Azure DDoS protection and IP allow-lists at the edge for administrative endpoints.

- All connectivity is verified from the running workload during release (private-endpoint and DNS checks are a release gate).

22. Package Deployment at the Client Site

Once developed, the platform is delivered to the client environment as a single, versioned release package. Because the UI, API, workers and infrastructure are built as independent artifacts against a shared contract, the packaged release is deployed into the client's own Azure subscription and Microsoft 365 tenant in a controlled, repeatable and reversible way.

22.1 What the Release Package Contains

Artifact	Description	Format
Infrastructure templates	Compiled Bicep + environment parameter files	ARM/Bicep + JSON
Web UI bundle	Production build of the SPA	Static assets
API image	Immutable API/BFF container	Container image (by digest)
Functions package	Workers and Durable orchestrations	Function app zip
Database migrations	Versioned schema and seed scripts	Migration bundle
Deployment scripts	Ordered install / upgrade / rollback	deploy/ scripts
Release manifest	Versions, digests, checksums, config map	Signed manifest
Runbooks & evidence	Install guide, validation, security evidence	docs/

22.2 Client-Side Prerequisites

- An Azure subscription in the approved sovereign region(s) with the required resource providers enabled.
- A licensed SharePoint Online tenant and the national SSO federation configured to Microsoft Entra ID.
- An authorised deployment identity with rights to create resources and role assignments in the target subscription.
- A private container registry (or access to the delivered registry) to hold the immutable API image.
- Network and firewall approvals for the single public ingress and the required Microsoft 365 / Graph egress endpoints.

22.3 Deployment Sequence

Deployment is a single orchestrated pipeline (or one scripted command) that runs the stages below in order. Each stage is idempotent and gated; a failure stops the release with a clear, recoverable state.

#	Stage	Action
1	Pre-flight	Validate prerequisites, quotas, identities and parameters; run Bicep what-if.
2	Provision infrastructure	Deploy Bicep to create/patch all resources, private endpoints and role assignments.
3	Configure identity & secrets	Bind Managed Identities; register the app; grant the SharePoint service-identity scope.
4	Database migration	Apply versioned schema migrations; verify with a rollback rehearsal.
5	Deploy API image	Roll out the API/BFF container by immutable digest to Container Apps.
6	Deploy workers	Publish the Functions package (Durable + timers + consumers).
7	Publish Web UI	Deploy the SPA bundle to Static Web Apps; bind the API base URL.
8	Apply gateway config	Import API definition and policies into API Management; configure Front Door/WAF.
9	Post-deploy validation	Synthetic checks: sign-in, capture, approval, promotion, search, audit.
10	Handover	Publish release manifest, dashboards, runbooks and support model.

22.4 Configuration Binding (no secrets in artifacts)

Artifacts are environment-agnostic. Environment-specific values — resource names, endpoints, the SharePoint site/library map, model deployment names — are supplied at deploy time through the parameter files and Key Vault references. No secret or environment value is baked into an image or bundle.

Configuration	Source at deploy time
Resource endpoints & names	Bicep parameter file per environment
Secrets / keys	Key Vault references resolved by Managed Identity
SharePoint site & library map	Admin configuration bound post-deploy

Configuration	Source at deploy time
Feature flags & cadences	Environment configuration store
Image version	Immutable digest pinned in the release manifest

22.5 Release Strategy, Validation & Rollback

- **Progressive release:** blue-green or canary rollout of the API and UI, shifting traffic only after health checks pass.
- **Automated validation:** synthetic transactions exercise the full capture → publish → search journey before traffic is fully shifted.
- **Rollback:** because artifacts are immutable and versioned, rollback re-points to the previous image digest / bundle and, if needed, runs the down-migration — no rebuild required.
- **Idempotent re-runs:** re-running the pipeline converges to the desired state; partial failures are safely retried.
- **Zero-downtime upgrades:** subsequent releases replace one package at a time behind the gateway, preserving availability.

22.6 Upgrades & Patching

- A new release is a new versioned manifest; the same pipeline promotes it through dev → staging → production.
- Independent packages allow a UI-only or API-only patch without redeploying the whole platform.
- Infrastructure drift is detected by re-running what-if; only reviewed changes are applied.

Packaged, repeatable delivery The client receives one signed, versioned release: infrastructure, UI, API, workers, migrations and runbooks. A single gated pipeline provisions the environment and deploys every package in order, validates the end-to-end journey, and can roll back to the previous version at any time.

23. Assumptions, Open Items & Next Steps

23.1 Open Items

#	Item	Owner
Q1	Batch cadence for the indexing pipeline (micro-batch vs. scheduled).	Architecture
Q2	Native versus integrated e-signature for signed approvals.	Product / Security
Q3	Customer-managed keys policy for encryption.	Security
Q4	Target container registry and image-signing approach at the client site.	Platform

23.2 Next Steps

1. Confirm document classes, sensitivity markings and the library map.
2. Finalise SSO federation and the SharePoint service-identity scope.
3. Stand up the VS Code workspace and package skeletons (web, api, functions, contracts, infra).
4. Provision the target environments via Infrastructure-as-Code and validate private connectivity.
5. Wire the CI/CD pipeline and produce the first signed release package.
6. Run an end-to-end pilot (capture → publish → search) before wider rollout.

Approval to build Endorsement of this Technical Architecture Document authorises detailed design and build against the solution and application architectures, the security, data and network models, the development and packaging methodology, and the packaged deployment approach defined herein.

— End of Document —

Appendices

The following appendices provide concrete, ready-to-adapt reference implementations for the continuous-integration and continuous-delivery pipelines and for the Infrastructure-as-Code composition root. They are illustrative baselines: placeholders in angle brackets are replaced with environment-specific values supplied through parameter files and pipeline variables. No secret or environment value is stored in source control.

Objective Give engineering and platform teams a starting, standards-aligned template for the build/test/scan pipeline, the gated release pipeline, and the Bicep composition root that provisions the whole environment with private networking and least-privilege identities.

Appendix A — CI/CD Pipeline (GitHub Actions)

Two workflows are provided. The pull-request workflow validates every change; the release workflow builds immutable artifacts, previews infrastructure with what-if, provisions the environment, applies database migrations, deploys each package, and runs post-deploy health checks. Both run under federated (secretless) OpenID Connect credentials to Azure.

A.1 Pull-Request Validation — .github/workflows/pr.yml

Purpose Fail fast on any change. Builds all packages, runs unit/integration/e2e/accessibility tests, and executes dependency, secret, SAST, container and IaC security scans before a merge is allowed.

```
# .github/workflows/pr.yml
name: pr-validation
on:
  pull_request:
    branches: [ main ]

permissions:
  contents: read
  id-token: write      # OIDC federation to Azure (no stored secrets)
  security-events: write # upload SAST/scan results

concurrency:
  group: pr-${{ github.ref }}
  cancel-in-progress: true

jobs:
  web:
    runs-on: ubuntu-latest
    defaults: { run: { working-directory: src/web } }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version-file: src/web/.nvmrc, cache: npm }
      - run: npm ci
      - run: npm run lint
      - run: npm run typecheck
      - run: npm run test:unit -- --coverage
      - run: npm run build
      - run: npx playwright install --with-deps && npm run test:e2e
      - run: npm run test:a11y      # axe accessibility gate

  api:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with: { global-json-file: global.json }
      - run: dotnet restore src/api/Docaya.Api.sln
      - run: dotnet format --verify-no-changes src/api/Docaya.Api.sln
      - run: dotnet build -c Release --no-restore src/api/Docaya.Api.sln
```

```
- run: dotnet test -c Release --no-build --collect:'XPlat Code Coverage'
- name: Verify OpenAPI contract is in sync
  run: dotnet run --project src/api/Docaya.Api -- --export-openapi | git diff --exit-code
```

```
security:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Dependency & secret scan
      run: |
        npm audit --audit-level=high || true
        gitleaks detect --no-banner --redact
    - name: SAST (CodeQL)
      uses: github/codeql-action/analyze@v3
    - name: Container image scan (Trivy)
      run: trivy fs --severity HIGH,CRITICAL --exit-code 1 .
    - name: IaC scan
      run: |
        az bicep build --file infra/main.bicep
        checkov -d infra --quiet
```

A.2 Release & Deploy — .github/workflows/release.yml

Purpose On a version tag, build immutable artifacts (UI bundle, API image by digest, Functions zip), preview infrastructure with what-if, provision the environment, migrate the database, deploy every package, then run synthetic health checks with a progressive (canary) rollout.

```
# .github/workflows/release.yml
name: release
on:
  push:
    tags: [ 'v*.*.*' ]    # semantic-version tag triggers a release

permissions: { contents: read, id-token: write, packages: write }

env:
  AZURE_REGION: <region>
  ACR_LOGIN_SERVER: <registry>.azurecr.io

jobs:
  build:
    runs-on: ubuntu-latest
    outputs: { api_digest: ${{ steps.push.outputs.digest }} }
    steps:
      - uses: actions/checkout@v4
      # --- Web UI bundle ---
      - uses: actions/setup-node@v4
        with: { node-version-file: src/web/.nvmrc, cache: npm }
      - run: cd src/web && npm ci && npm run build
      - uses: actions/upload-artifact@v4
        with: { name: web-bundle, path: src/web/dist }
      # --- Functions package ---
      - uses: actions/setup-dotnet@v4
        with: { global-json-file: global.json }
      - run: dotnet publish src/functions -c Release -o out/functions
      - run: cd out/functions && zip -r ../functions.zip .
      - uses: actions/upload-artifact@v4
        with: { name: functions-zip, path: out/functions.zip }
      # --- API container image (pushed by immutable digest) ---
      - uses: azure/login@v2
        with:
          client-id: ${{ vars.AZURE_CLIENT_ID }}
          tenant-id: ${{ vars.AZURE_TENANT_ID }}
          subscription-id: ${{ vars.AZURE_SUBSCRIPTION_ID }}
      - run: az acr login -n ${{ env.ACR_LOGIN_SERVER }}
      - id: push
      run: |
        IMG=${{ env.ACR_LOGIN_SERVER }}/docaya-api:${{ github.ref_name }}
```

```

docker build -t $IMG src/api
docker push $IMG
echo "digest=$(docker inspect --format='{{index .RepoDigests 0}}' $IMG)" >> $GITHUB_OUTPUT

```

```

deploy:
  needs: build
  runs-on: ubuntu-latest
  environment: production    # requires reviewer approval
  steps:
    - uses: actions/checkout@v4
    - uses: azure/login@v2
    with:
      client-id: ${{ vars.AZURE_CLIENT_ID }}
      tenant-id: ${{ vars.AZURE_TENANT_ID }}
      subscription-id: ${{ vars.AZURE_SUBSCRIPTION_ID }}
    # 1) Preview then provision infrastructure
    - name: what-if (no mutation)
      run: |
        az deployment sub what-if --location $AZURE_REGION \
          --template-file infra/main.bicep \
          --parameters infra/main.parameters.prod.json
    - name: provision
      run: |
        az deployment sub create --location $AZURE_REGION \
          --template-file infra/main.bicep \
          --parameters infra/main.parameters.prod.json \
          apiImageDigest='${{ needs.build.outputs.api_digest }}'
    # 2) Database migration (with rollback rehearsal in staging)
    - run: dotnet tool run dotnet-ef database update --project src/api/Docaya.Infrastructure
    # 3) Deploy packages
    - uses: actions/download-artifact@v4
    - name: deploy web + functions
      run: |
        az staticwebapp deploy -n <swa-name> --source ./web-bundle
        az functionapp deployment source config-zip -g <rg> -n <func> --src functions.zip
    # 4) Post-deploy health (canary before full shift)
    - name: synthetic health checks
      run: ./deploy/smoke.sh --canary 10 --checks signin,capture,approve,promote,search,audit

```

Appendix B — Sample Infrastructure-as-Code (Bicep)

The composition root deploys the whole environment at subscription scope, in dependency order, wiring private networking, least-privilege identities and diagnostics. Each concern is a module; the root passes shared context (VNet, subnets, Key Vault, Log Analytics) between them.

B.1 Composition Root — `infra/main.bicep`

Purpose One deployment provisions network, identity, secrets, data stores, messaging, observability, compute and the edge — then assigns resource-scoped data-plane roles. No portal clicks; fully reproducible.

```
// infra/main.bicep
targetScope = 'subscription'

@description('Deployment environment') @allowed(['dev','staging','prod'])
param env string
param location string
@description('Immutable API image digest supplied by the release pipeline')
param apiImageDigest string
param tags object = { workload: 'docaya', env: env }

var rgName = 'rg-docaya-${env}'

resource rg 'Microsoft.Resources/resourceGroups@2023-07-01' = {
  name: rgName
  location: location
  tags: tags
}

// 1) Network first (VNet, subnets, NSGs, private DNS)
module network 'modules/network.bicep' = {
  name: 'network', scope: rg
  params: { env: env, location: location, tags: tags }
}

// 2) User-assigned managed identities
module identity 'modules/identity.bicep' = {
  name: 'identity', scope: rg
  params: { env: env, location: location, tags: tags }
}

// 3) Observability (Log Analytics + App Insights)
module obs 'modules/observability.bicep' = {
  name: 'observability', scope: rg
  params: { env: env, location: location, tags: tags }
}

// 4) Key Vault (private endpoint)
module kv 'modules/keyvault.bicep' = {
  name: 'keyvault', scope: rg
  params: {
    env: env, location: location, tags: tags
    subnetId: network.outputs.peSubnetId
    privateDnsZoneId: network.outputs.kvDnsZoneId
  }
}

// 5) Data & messaging (all private-endpoint, public access disabled)
module data 'modules/data.bicep' = {
  name: 'data', scope: rg
  params: {
    env: env, location: location, tags: tags
    peSubnetId: network.outputs.peSubnetId
    dnsZones: network.outputs.dataDnsZones
    logAnalyticsId: obs.outputs.workspaceId
  }
}

// 6) Compute: Container Apps env (API + workers) by immutable digest
module compute 'modules/containerapp.bicep' = {
```

```

name: 'compute', scope: rg
params: {
  env: env, location: location, tags: tags
  appSubnetId: network.outputs.appSubnetId
  identityId: identity.outputs.apiIdentityId
  apiImage: '${data.outputs.acrLoginServer}/docaya-api@${apiImageDigest}'
  appInsightsConnDup: obs.outputs.appInsightsConnectionString
  keyVaultUri: kv.outputs.vaultUri
}
}

// 7) Edge & gateway: APIM, Front Door + WAF, Static Web App
module edge 'modules/edge.bicep' = {
  name: 'edge', scope: rg
  params: {
    env: env, location: location, tags: tags
    apiFqdn: compute.outputs.apiInternalFqdn
  }
}

output apiUrl string = edge.outputs.frontDoorEndpoint
output webUrl string = edge.outputs.staticWebAppUrl

```

B.2 Example Module — modules/containerapp.bicep (excerpt)

Purpose Run the API and workers on Azure Container Apps with a user-assigned managed identity, VNet integration, autoscale rules and diagnostics — pulling the image by digest, never a floating tag.

```

// modules/containerapp.bicep (excerpt)
param env string
param location string
param tags object
param appSubnetId string
param identityId string
param apiImage string // pinned by digest
param keyVaultUri string
param appInsightsConnDup string

resource cae 'Microsoft.App/managedEnvironments@2024-03-01' = {
  name: 'cae-docaya-${env}'
  location: location
  tags: tags
  properties: {
    vnetConfiguration: { infrastructureSubnetId: appSubnetId, internal: true }
    zoneRedundant: env == 'prod'
  }
}

resource api 'Microsoft.App/containerApps@2024-03-01' = {
  name: 'ca-docaya-api-${env}'
  location: location
  tags: tags
  identity: { type: 'UserAssigned', userAssignedIdentities: { '${identityId}': {} } }
  properties: {
    managedEnvironmentId: cae.id
    configuration: {
      activeRevisionsMode: 'Multiple' // enables blue-green / canary
      ingress: { external: false, targetPort: 8080, transport: 'auto' }
    }
  }
  template: {
    containers: [ {
      name: 'api'
      image: apiImage
      env: [
        { name: 'KEYVAULT_URI', value: keyVaultUri }
        { name: 'APPLICATIONINSIGHTS_CONNECTION_STRING', value: appInsightsConnDup }
      ]
      resources: { cpu: json('1.0'), memory: '2Gi' }
    } ]
  }
  scale: {
    minReplicas: env == 'prod' ? 3 : 1
    maxReplicas: 30
  }
}

```

```

    rules: [ { name: 'http', http: { metadata: { concurrentRequests: '80' } } } ]
  }
}
}
}

output apiInternalFqdn string = api.properties.configuration.ingress.fqdn

```

B.3 Environment Parameters — infra/main.parameters.prod.json

Purpose Environment-specific, non-secret values. Secrets are never placed here — they are Key Vault references resolved at runtime by the managed identity.

```

{
  "$schema": "https://schema.management.azure.com/schemas/2019-04-01/deploymentParameters.json#",
  "contentVersion": "1.0.0.0",
  "parameters": {
    "env": { "value": "prod" },
    "location": { "value": "<region>" },
    "apiImageDigest": { "value": "" } // injected by the release pipeline
  }
}

```

B.4 Deploy Commands

```

# validate & preview (no mutation)
az bicep build --file infra/main.bicep
az deployment sub what-if --location <region> \
  --template-file infra/main.bicep \
  --parameters infra/main.parameters.prod.json

# provision (authorised deployment identity)
az deployment sub create --location <region> \
  --template-file infra/main.bicep \
  --parameters infra/main.parameters.prod.json \
  apiImageDigest='<digest-from-pipeline>'

```

Result Running the release pipeline against these templates provisions a complete, private, zone-redundant environment and deploys every package by immutable version — repeatably, reviewably and reversibly.

— End of Appendices —